Persistent Cookies and DNS Rebinding Redux ha.ckers.org web application security lab

Persistent Cookies and DNS Rebinding Redux ha.ckers.org web application security lab - Author not stated, ha.ckers.org.

- Published: date not stated
- Original: <http://ha.ckers.org/blog/20090120/persistent-cookies-and-dns-rebinding-redux/>
- Preserved from: http://ha.ckers.org/blog/20090120/persistent-cookies-and-dns-rebinding-redux/ (stored) on 2026-08-09
- Capture timestamp: 20090122095144
- Licence: unknown

Content

> UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Persistent Cookies and DNS Rebinding Redux ha.ckers.org web application security lab

[[image: image]](http://ha.ckers.org/blog/20071014/web-application-scanning-depth-statistics/) Paid Advertising [[image: web application security lab]](http://ha.ckers.org/)

Persistent Cookies and DNS Rebinding Redux

In an attempt to clarify my post on the dangers associated with persistent cookies and DNS rebinding, I'd like to give a simple scenario and then describe solutions. Let's say there is an intranet website called intranet.exploitable.com that resolves to 10.10.10.10, and there is an attacker website called www.attacker.com that resolves to 222.222.222.222. Now let's say intranet.exploitable.com typically sets a cookie that has also has a known XSS vulnerability in it (could be known because the attacker knows what sort of open source software is used internally, or they were once a contractor or whatever...). Now let's also assume that the website is not SSL, as most aren't and it would mess up the attack with a mis-match SSL error.

Okay, so the victim user visits www.attacker.com who sets the same cookie as something like this:

> Set-Cookie: last-visited=<script>alert("XSS")</script>; path=/

Then the user shuts down their browser, or the attacker forces a browser shutdown through any one of the dozens of browser DoS scripts out there. Eventually the user goes back to www.attacker.com, but this time, the site changes it's DNS to point to 10.10.10.10. Because the browser was shut down, the DNS for www.attacker.com is now allowed to be rebound to the new

IP address, which happens to be the IP address of intranet.exploitable.com. The user now visits that site with the XSS exploit in their cookies, with the incorrect host header:

> Host: www.attacker.com

However, because most sites don't care about host headers, the request is still parsed by intranet.exploitable.com's website. The XSS is now running there. While this wouldn't allow the attacker to log into their account, it would allow them to "see" what is running on the victim's intranet website, by using an XSS shell. Although this attack may take a while, it's not that difficult, compared to a lot of other rebinding attacks.

Now in terms of mitigation, there's a whole host of things you can do if you happen to run intranet.exploitable.com. Firstly, using SSL would stop this attack because of the SSL to hostname mis-match. Secondly, not allowing any unknown host header to be sent would stop the incorrect host header from being processed. Using client side protections like LocalRodeo would stop the intranet from being contacted as well. Lastly, making sure that *all* cookies are removed upon each shut down of the browser would stop the attacker from being able to re-use their cookies after having forced the victim's browser to shut down. I hope all that was a lot more clear.
